KAŞ BÖLGESİ TABANLI
DEEPFAKE TESPİTİ
Swin V2-Tiny + LBP + GLCM + Gabor + Wavelet Fusion
Notebook Kod Açıklama ve Teknik İnceleme Raporu

İncelenen dosya:
Eyebrow_SwinV2_LBP_GLCM_Gabor_Wavelet_Fusion_Nazlican_v2 (1).ipynb

Rapor tarihi: 7 Ağustos 2026

İçindekiler

1. Raporun amacı ve kapsamı
2. Notebookun genel amacı
3. Uçtan uca iş akışı
4. Çalışma ortamı ve local source yapısı
5. Merkezi konfigürasyon
6. Tekrarlanabilirlik, kayıt ve deney klasörü
7. Güvenli dosya yazma mekanizması
8. Metadata hazırlama ve veri doğrulama
9. Video kimliği ve veri sızıntısı kontrolleri
10. El yapımı özellik çıkarımı
11. Özellik önbelleği ve ölçekleme
12. PyTorch Dataset ve DataLoader yapısı
13. Swin V2-Tiny + texture fusion mimarisi
14. Eğitim stratejisi
15. Eşik seçimi, metrikler ve checkpointler
16. Smoke test
17. Frame ve video düzeyi değerlendirme
18. Üretilen grafikler ve dosyalar
19. Kalite kapıları ve çıktı manifesti
20. Ablation çalışma seçenekleri
21. Teknik güçlü yönler
22. Sınırlılıklar ve geliştirme önerileri
23. Hücre bazlı özet
24. Sonuç

1. Raporun amacı ve kapsamı

Bu rapor, yüklenen Jupyter/Google Colab notebookunun ne yaptığını; veri girişinden özellik çıkarımına, model eğitiminden değerlendirme ve deney çıktılarının kaydedilmesine kadar ayrıntılı biçimde açıklamak için hazırlanmıştır. İnceleme, notebook kaynak kodunun statik analizi üzerinden yapılmıştır.

Önemli kapsam notu

Yüklenen notebookta kayıtlı hücre çıktıları bulunmamaktadır. Bu nedenle rapor performans sonuçlarını doğrulamaz; kodun amaçlanan davranışını, veri akışını, güvenlik kontrollerini ve teknik tasarımını açıklar.

Notebook yalnızca daha önce kaş bölgesine göre kırpılmış ROI framelerini kullanır. Yüz tespiti, yüz işaret noktası çıkarımı veya yeniden kaş kırpma işlemi içermez. Bu ayrım önemlidir: kodun görevi ROI üretmek değil, mevcut kaş görüntülerinden görsel ve tekstürel özellikler çıkararak real/fake sınıflandırması yapmaktır.

2. Notebookun genel amacı

Temel amaç, kaş bölgesinin deepfake tespitinde taşıdığı bilgiyi iki farklı temsil biçimini birleştirerek kullanmaktır:

☐ Derin görüntü temsili: ImageNet ağırlıklarıyla başlatılabilen Swin V2-Tiny backbone.
☐ El yapımı tekstür temsili: LBP, GLCM, Gabor ve Wavelet yöntemlerinden oluşan 240 boyutlu özellik vektörü.
☐ Fusion sınıflandırıcı: İki kolu ortak gizli uzaya projekte edip birleştiren ve tek real/fake logiti üreten MLP.

Bu nedenle kod klasik bir yalnız-CNN deneyi veya yalnız-SVM deneyi değildir. Görüntü kolu ile matematiksel tekstür özelliklerini aynı sinir ağı mimarisinde bir araya getiren hibrit bir füzyon yaklaşımıdır.

3. Uçtan uca iş akışı

Şekil 1. Notebookun uçtan uca işlem sırası.

1. local source package bağlanır ve gerekli Python paketleri kurulur.

2. Proje, kaş verisi, metadata ve sonuç klasörlerinin varlığı doğrulanır.
3. Merkezi CONFIG sözlüğü üzerinden deney parametreleri yüklenir.
4. Seed, cihaz, run kimliği, klasörler, loglar ve ortam bilgileri hazırlanır.
5. Kaş metadata ile seçim metadata okunur, normalize edilir ve eşleştirilir.
6. Video kimlikleri geri kazanılır; train/val/test video sızıntısı kontrol edilir.
7. Her kaş ROI için LBP, GLCM, Gabor ve Wavelet özellikleri çıkarılır.
8. Özellikler cache'e kaydedilir ve yalnızca train split üzerinde StandardScaler eğitilir.
9. PyTorch Dataset, görüntü ve texture özelliklerini aynı örnek içinde sunar.
10. Swin V2 ve texture kolları eğitilir; en iyi model validation ROC-AUC ile seçilir.
11. Validation F1'e göre karar eşiği belirlenir ve test tahminleri üretilir.
12. Frame ve video düzeyi metrikler, grafikler, checkpointler ve manifest kaydedilir.

4. Çalışma ortamı ve local source package yapısı

4.1 Colab kurulumu

İlk kod bloğu local source package'ı Deney1_Source_Runtime konumuna bağlar ve deney için gerekli kütüphaneleri
kurar. Kurulan ana paketler PyTorch, TorchVision, scikit-image, PyWavelets, scikit-learn, pandas, NumPy, Matplotlib, Pillow, PyYAML, tqdm, joblib ve OpenCV'dir.

```
from google.colab import local source
local source.mount("Deney1_Source_Runtime")
!pip -q install "torch>=2.3" "torchvision>=0.18" ...
```


4.2 Zorunlu yollar

Bileşen	Kodun beklediği konum
Proje kökü	Deney1_Source/AISC DeepFake Çalışmaları
Deney kökü	Deneyler/Nazlıcan/Deney 1
Kaş ROI klasörü	Deney 1/Kaş
Kaş metadata	Deney 1/Kaş/metadata.csv
Seçim metadata	Deneyler/Deney 1 Frame/secim_metadata.csv
Sonuç kökü	Deney 1/Sonuçlar

Preflight bloğu bu yolların tamamını çalışmanın başında kontrol eder. Herhangi bir yol eksikse eğitim başlamadan FileNotFoundError üretir. Böylece saatler sürebilecek bir eğitim, yanlış local source yolu nedeniyle geç aşamada boşa gitmez.

5. Merkezi konfigürasyon

CONFIG sözlüğü deney davranışının tek merkezi ayar kaynağıdır. Veri sütunları, görüntü boyutu, özellik parametreleri, eğitim hiperparametreleri ve çalışma kontrol seçenekleri bu blokta tutulur.

Ayar	Değer	İşlev
Seed	42	Python, NumPy ve PyTorch tekrar üretilebilirliği
Görüntü boyutu	224 × 224	Swin ve özellik çıkarımı için ortak ROI boyutu
Pretrained	True	Swin V2-Tiny varsayılan ağırlıkları
Backbone dondurma	5 epoch	İlk aşamada yalnız projection/fusion başlığı eğitilir
Batch size	16	Train için veri boyutuna göre güvenli biçimde ayarlanır
Maksimum epoch	30	Early stopping daha erken durdurabilir
Head öğrenme oranı	2e-4	Projection ve sınıflandırıcı katmanları
Backbone öğrenme oranı	2e-5	Açılan Swin katmanları için daha düşük LR
Weight decay	1e-4	AdamW düzenlileştirmesi
Dropout	0.30	Projection ve classifier katmanlarında
Hidden dimension	256	Her kolun ortak temsil boyutu
Patience	8	Validation ROC-AUC iyileşmezse erken durdurma
AMP	True	CUDA üzerinde mixed precision

Ayar	Değer	İşlev
Gradient clip	1.0	Gradyan patlamasına karşı norm sınırı
Varsayılan mod	full	Swin + 240 texture özelliği

5.1 Eşik taraması

Karar eşiği 0.10 ile 0.90 arasında 161 noktada taranır. Aralık uçları dahil olduğundan adım büyüklüğü 0.005'tir. Her epoch için validation F1 değerini en yüksek yapan eşik bulunur.

6. Tekrarlanabilirlik, kayıt ve deney klasörü

6.1 Seed yönetimi

seed_everything fonksiyonu PYTHONHASHSEED, random, NumPy, CPU PyTorch ve CUDA seed

değerlerini aynı sayıya ayarlar. cuDNN benchmark kapatılır ve deterministic mod etkinleştirilir. Bu ayarlar aynı ortamda sonuçların daha tutarlı tekrarlanmasını amaçlar.

6.2 Run kimliği ve klasörler

Her çalıştırma için zaman damgası ve seed içeren bir run kimliği oluşturulur. Sonuçlar bu run klasöründe checkpoints, logs, metrics, predictions, figures ve artifacts alt klasörlerine ayrılır.

<YYYYMMDD_HHMM>_eyebrow_swinv2_lbp_glcm_gabor_wavelet_seed42/
checkpoints/ ├──
logs/ ├──
metrics/ ├──
predictions/ ├──
figures/ ├──
artifacts/ └──

6.3 Deney ortamının kaydedilmesi

☐ config_resolved.yaml: Çalışmada gerçekten kullanılan ayarlar, run_id, cihaz ve oluşturulma zamanı.

☐ environment.json: Python, işletim sistemi, CUDA, GPU adı ve paket sürümleri.

☐ requirements_lock.txt: pip freeze çıktısı; ortamın daha sonra yeniden kurulabilmesi için.

☐ pipeline.log: İşlem adımları ve epoch bilgileri.

Teknik gözlem

resume_run_id aynı klasörü yeniden hedeflemeyi sağlar; fakat notebookta last.ckpt dosyasını otomatik yükleyerek epoch, optimizer ve scheduler durumundan devam eden gerçek bir resume mekanizması bulunmamaktadır. Bu seçenek klasörü/cache'i yeniden kullanır, ancak eğitimi kaldığı epoch'tan otomatik sürdürmez.

7. Güvenli dosya yazma mekanizması

Notebook kritik çıktıları doğrudan hedef dosyaya yazmak yerine önce .tmp uzantılı geçici dosyaya yazar. Yazma tamamlandıktan ve mümkünse geri yükleme testi yapıldıktan sonra os.replace ile atomik biçimde hedefe taşır.

Fonksiyon	Koruduğu çıktı	Doğrulama
atomic_write_bytes/text	JSON, CSV, metin	flush + fsync + atomik replace
atomic_joblib_dump	Scaler ve feature cache	Geçici dosya joblib.load ile geri okunur
atomic_torch_save	Checkpoint	Geçici checkpoint yüklenir; gerekli anahtarlar kontrol edilir
sha256_file	Manifest dosya özeti	Dosya parça parça SHA-256 ile hashlenir

Bu tasarım Colab bağlantısı kesilmesi, local source senkronizasyon gecikmesi veya işlem ortasında durma gibi durumlarda yarım yazılmış dosyaların kalma ihtimalini azaltır.

8. Metadata hazırlama ve veri doğrulama

8.1 Okunan iki metadata kaynağı

☐ Kaş metadata: ROI üretim sürecinin sample_id, output_path, output_relative_path, input_path, label, split ve status alanları.

☐ Seçim metadata: Orijinal frame seçimini ve frame'in geldiği video yolunu taşıyan sinif, split, orijinal_yol, yeni_yol ve dosya_adi alanları.

8.2 Normalizasyon

Status değerleri büyük harfe, label ve split değerleri küçük harfe çevrilir. training train, → validation/valid val ve testing test eşlemeleri uygulanır. Bu işlem yazım biçimi farklılıklarının → → yanlış filtrelemeye yol açmasını önler.

8.3 Başarılı ROI filtreleme

Yalnız SUCCESS veya OK durumundaki, label değeri real/fake olan ve split değeri train/val/test olan satırlar tutulur. Resolved image path fiziksel olarak mevcut değilse satır kullanılmaz; kayıp başarılı dosya sayısı ayrıca raporlanır.

8.4 Metadata eşleştirme

ROI satırları ile seçim metadata satırları frame_name üzerinden many-to-one doğrulamalı left merge ile eşleştirilir. Merge başarısı pandas'ın _merge göstergesiyle kontrol edilir. Eşleşmeyen satırlar metadata_unmatched_rows.csv dosyasına yazılır ve süreç durdurulur.

Veri kalitesi ayrımı

Eşleşme gerçekleşmiş olsa bile source_video_path boşsa bu durum eşleşme hatası değil, kaynak metadata kalite hatası olarak ayrı dosyaya yazılır. Böylece iki farklı problem türü birbirine karıştırılmaz.

9. Video kimliği ve veri sızıntısı kontrolleri

Deepfake frame çalışmalarında en kritik risklerden biri, aynı videodan gelen farklı framelerin hem eğitim hem test kümesine düşmesidir. Kod bu riski azaltmak için seçim metadata içindeki orijinal video yolunun parent klasör adını geri kazanır ve video_id üretir.

```
video_id = label + "__" + source_video_name
# Örnek: fake__FaceSwap_477_487
```

☐ Her sample_id benzersiz olmalıdır.

☐ Bir video_id yalnızca tek split içinde bulunabilir.
☐ Bir video_id altında yalnızca tek target bulunabilir.
☐ Train, validation ve test video kümeleri ikili olarak ayrık olmalıdır.
☐ ROI metadata label/split değerleri seçim metadata ile birebir uyuşmalıdır.
Kontroller başarıyla tamamlandığında kullanılan tüm satırlar artifacts/metadata_used.csv dosyasına yazılır; split, sınıf ve video sayıları metrics/data_accounting.json içinde tutulur.
Varsayım
video_id, kaynak yolun yalnızca parent klasör adına dayanır. Aynı sınıf içinde farklı dizinlerde aynı parent klasör adı kullanılırsa kimlik çakışması olabilir. Veri yapısında parent klasör adlarının benzersiz olması beklenmektedir.
10. El yapımı özellik çıkarımı
Her ROI önce gri tona çevrilir ve en-boy oranı korunarak 224×224 alana siyah padding ile yerleştirilir. Ardından dört özellik ailesi çıkarılır ve tek float32 vektörde birleştirilir.

Özellik grubu	Boyut	Hesaplama	Kaş bölgesindeki anlamı
LBP	54	10 + 18 + 26 histogram bin'i	Yerel kıl/doku mikro örüntüleri
GLCM	72	6 özellik × 3 mesafe × 4 açı	Gri seviye komşuluk ve doku düzeni
Gabor	72	3 frekans × 6 yön × 4 istatistik	Kıl yönü, çizgisel yapı ve frekans
Wavelet	42	7 alt bant × 6 istatistik	Çok ölçekli kenar ve yüksek frekans artefaktları
Toplam	240	Dört grubun concatenation'ı	Fusion texture kolunun girdisi

10.1 LBP - Local Binary Pattern
Uniform LBP üç ölçekte hesaplanır: (R=1,P=8), (R=2,P=16) ve (R=3,P=24). Uniform yöntemde histogram bin sayısı P+2'dir. Bu nedenle 10 + 18 + 26 = 54 özellik oluşur. Histogram her ölçekte toplamı 1 olacak şekilde normalize edilir.
10.2 GLCM - Gray Level Co-occurrence Matrix
Görüntü 256 gri seviyeden 32 seviyeye kuantize edilir. Simetrik ve normalize GLCM; 1, 2 ve 4 piksel mesafelerinde, 0°, 45°, 90° ve 135° açılarında hesaplanır. Her kombinasyondan contrast, dissimilarity, homogeneity, energy, correlation ve ASM değerleri alınır: 3 × 4 × 6 = 72.
10.3 Gabor özellikleri
Gabor filtreleri 0.10, 0.20 ve 0.30 frekanslarında; 0°, 30°, 60°, 90°, 120° ve 150° yönlerinde uygulanır.

Her filtre yanıtının karmaşık büyüklüğü üzerinden mean, standard deviation, energy ve histogram entropy hesaplanır: 3 × 6 × 4 = 72.
10.4 Wavelet özellikleri
Daubechies-2 (db2) ile iki seviyeli 2B ayrıştırma yapılır. Bir yaklaşım bandı (LL) ve iki seviyedeki üçer detay bandı (LH, HL, HH) olmak üzere toplam yedi bant vardır. Her bant için mean, standard deviation, energy, absolute mean, absolute maximum ve entropy çıkarılır: 7 × 6 = 42.
10.5 Sayısal güvenlik
Birleşik vektörde NaN veya sonsuz değer bulunursa FloatingPointError oluşur. İlk geçerli örnekte özellik boyutu hesaplanır ve tam olarak 240 olması assert ile doğrulanır. Özellik adları ve grup slice aralıkları artifacts klasörüne yazılır.
Padding etkisi
En-boy oranını koruyan padding geometrik bozulmayı azaltır; ancak siyah sınırlar modele veya texture ölçülerine yapay bir ipucu verebilir. Farklı ROI boyutları varsa padding oranının sınıflarla korelasyon göstermediği ayrıca denetlenmelidir.
11. Özellik önbelleği ve ölçekleme
11.1 Cache sürümleme
Özellik parametreleri, bölge, veri kökü, metadata yolu ve görüntü boyutu JSON biçiminde hashlenerek 12 karakterlik CACHE_VERSION üretilir. Cache dosyası bu sürüm ile adlandırılır. Aynı run klasöründe aynı ayarlarla yeniden çalıştırmada hazır örnekler tekrar hesaplanmaz.
11.2 Periyodik güvenli kayıt
Feature çıkarımı sırasında cache her 100 örnekte bir atomik olarak kaydedilir. Hatalı örnekler metrics/texture_feature_errors.csv dosyasına yazılır; yalnız cache içinde başarıyla bulunan örnekler sonraki aşamaya geçer.

11.3 Train-only StandardScaler

StandardScaler yalnızca train split'in 240 boyutlu vektörleri üzerinde fit edilir. Validation ve test verileri aynı train istatistikleriyle dönüştürülür. Bu uygulama, validation/test ortalama ve standart sapmalarının eğitime sızmasını engeller.

☐ texture_scaler.joblib: Fit edilmiş scaler.

☐ texture_feature_schema.json: Boyut, cache sürümü ve grup indeksleri.

☐ texture_features_train/val/test.npz: sample_id, video_id, label, ham ve ölçeklenmiş vektörler.

Cache geçerliliği

Cache anahtarında görüntü dosyalarının içerik hash'i veya değiştirilme zamanı bulunmaz. Aynı sample_id ve aynı yolda görüntü içeriği değişirse eski cache kullanılabilir. Böyle bir durumda overwrite_feature_cache=True yapılmalıdır.

12. PyTorch Dataset ve DataLoader yapısı

12.1 Görüntü kolu

RGB ROI, ImageOps.pad ile 224×224'e getirilir; ardından tensor'a çevrilir ve ImageNet mean/std değerleriyle normalize edilir. Geometrik augmentation uygulanmaz. Kodun yorumu, önceden hesaplanan texture özellikleri ile görüntü kolunun aynı değişmemiş ROI'yi temsil etmesini gerekçe gösterir.

12.2 Dataset çıktısı

Alan	Tür/şekil	Amaç
image	Float tensor [3,224,224]	Swin kolunun girdisi
texture	Float tensor [D]	Mod seçimine göre texture özellikleri
target	Float scalar	0=real, 1=fake
sample_id	String	Tahmin ve cache takibi
video_id	String	Video düzeyi toplulaştırma
image_path / source_video_path	String	İzlenebilirlik

12.3 DataLoader güvenliği

Train batch size, train örnek sayısından büyükse otomatik küçültülür. Train seti en az iki örnek içermelidir. Son batch yalnızca bir örnek kalacaksa drop_last etkinleştirilir; böylece tek örnekli batch kaynaklı normalizasyon sorunları önlenirken küçük veri setinde loader'ın tamamen boş kalması engellenir.

13. Swin V2-Tiny + texture fusion mimarisi

Şekil 2. Hibrit iki kollu model mimarisi.

13.1 Swin kolu

torchvision.models.swin_v2_t oluşturulur. Orijinal sınıflandırma head'i Identity ile değiştirilir ve backbone'un çıktı boyutu dinamik olarak head.in_features üzerinden alınır. Derin özellikler LayerNorm, Linear(hidden_dim=256), GELU ve Dropout katmanlarından geçirilir.

13.2 Texture kolu

Seçilen el yapımı özellik vektörü LayerNorm, Linear(256), GELU ve Dropout ile 256 boyutlu ortak temsile dönüştürülür.

13.3 Fusion classifier

Full modda iki 256 boyutlu kol birleştirilerek 512 boyutlu temsil elde edilir. Classifier sırasıyla Linear 256, LayerNorm, GELU, Dropout, Linear 128, GELU, Dropout ve Linear 1 katmanlarından→ → → oluşur. Son değer logittir; olasılık değerlendirme sırasında sigmoid ile elde edilir.

Mod	Swin	Texture grupları	Texture D	Fusion girişi
swin_only	Var	Yok	0	256
texture_only	Yok	LBP+GLCM+Gabor+W		
avelet	240			256
swin_lbp	Var	LBP	54	512
swin_lbp_glcm	Var	LBP+GLCM	126	512
swin_lbp_glcm_gabor	Var	LBP+GLCM+Gabor	198	512
full	Var	Tüm gruplar	240	512

14. Eğitim stratejisi

14.1 İki aşamalı fine-tuning

İlk beş epoch boyunca Swin backbone tamamen dondurulur. Projection ve classifier katmanları öğrenir. Beşinci epoch sonrasında yalnız Swin'in son feature stage'i ve final norm katmanı açılır. Böylece önce yeni başlık istikrarlı hale gelir, ardından derin temsil kontrollü biçimde veriye uyarlanır.

14.2 Sınıf ağırlıklı loss

Train hedeflerinden balanced class weight hesaplanır. Positive class için oran BCEWithLogitsLoss(pos_weight=...) içine verilir. Böylece sınıf sayılarındaki dengesizlik fake sınıf kaybının ağırlığını otomatik ayarlar.

14.3 Optimizer ve scheduler

☐ AdamW optimizer kullanılır.

☐ Backbone parametrelerine 2e-5, başlık parametrelerine 2e-4 öğrenme oranı atanır.

☐ CosineAnnealingLR öğrenme oranını epochlar boyunca azaltır.

☐ Backbone açıldığında optimizer ve scheduler yeniden kurulur.

Optimizer yeniden başlatma etkisi
Backbone açıldığı anda optimizer yeniden oluşturulduğu için önceki AdamW momentum/ikinci moment durumları korunmaz. Bu bilinçli bir sadeleştirme olabilir; ancak kesintisiz optimizer durumuyla karşılaştırılması yararlı olabilir.
14.4 AMP ve gradyan kontrolü
CUDA varsa autocast ve GradScaler ile mixed precision kullanılır. Backpropagation sonrasında gradyanlar unscale edilir ve global norm 1.0 ile kırpılır. Loss sonlu değilse eğitim hemen durdurulur.
15. Eşik seçimi, metrikler ve checkpointler
15.1 Hesaplanan metrikler
☐ Accuracy
☐ Balanced accuracy
☐ Precision
☐ Recall
☐ F1
☐ ROC-AUC
☐ Average precision

15.2 Model ve eşik seçiminin ayrılması
En iyi checkpoint validation ROC-AUC değerine göre seçilir. Buna karşılık o checkpoint için kullanılacak karar eşiği, aynı epoch validation tahminlerinde F1'i en yüksek yapan eşiktir. Bu ayrım faydalıdır: ROC-AUC sıralama performansını, F1-optimal eşik ise binary karar noktasını belirler.
Grafik yorumlama uyarısı
Train F1 sabit 0.50 eşikle; validation F1 ise her epoch yeniden optimize edilen eşikle hesaplanır. Bu nedenle
eğitim ve validation F1 eğrileri birebir aynı karar eşiğine dayalı değildir. Aralarındaki fark doğrudan overfitting ölçüsü olarak yorumlanmamalıdır.
15.3 Checkpoint içeriği
☐ Model, optimizer, scheduler ve scaler state_dict değerleri
☐ Epoch, best metric ve best threshold
☐ Tüm eğitim geçmişi ve çözülmüş CONFIG
☐ Feature slice bilgileri
☐ Python, NumPy, PyTorch ve CUDA RNG durumları
Her epoch last.ckpt güncellenir; ROC-AUC iyileştiğinde best.ckpt yazılır. Validation AUC hesaplanamazsa last.ckpt defansif olarak best.ckpt yerine kopyalanır.
16. Smoke test
Tam eğitim öncesinde iki train ve iki validation batch'i üzerinde kısa bir smoke test çalıştırılır. Amaç modelin yüksek performansını ölçmek değil; veri akışı, tensor boyutları, loss, optimizer, AMP ve checkpoint yazma/okuma mekanizmasının çalıştığını doğrulamaktır.
☐ Smoke testte pretrained=False kullanılır; gereksiz ağırlık yükleme önlenir.
☐ Geçici smoke_test.ckpt yazılır, geri okunur ve silinir.
☐ GPU belleği ve Python nesneleri temizlenir.
17. Frame ve video düzeyi değerlendirme
17.1 Frame düzeyi
Best checkpoint test DataLoader üzerinde çalıştırılır. Her örnek için sample_id, video_id, görüntü yolu, kaynak video yolu, target ve probability_fake kaydedilir. Seçilen validation eşiğine göre prediction ve correct alanları eklenir.
17.2 Video düzeyi
Frame tahminleri video_id bazında gruplanır. Her video için fake olasılıklarının aritmetik ortalaması alınır; aynı seçilmiş eşik kullanılarak video sınıfı üretilir. Bir video altında birden fazla target varsa süreç durdurulur.

Toplulaştırma tercihi
Mean probability basit ve açıklanabilir bir yöntemdir; ancak videolardaki frame sayısı ve artefaktların zamansal dağılımı dikkate alınmaz. Median, top-k mean, temporal model veya güven ağırlıklı agregasyon ek deneyler olabilir.
18. Üretilen grafikler ve dosyalar
Dosya Ne gösterir?
dataset_distribution.png Train/val/test içinde real ve fake örnek sayıları
training_loss_curve.png Train ve validation loss değişimi
training_f1_curve.png Train ve validation F1 değişimi
test_confusion_matrix_frame_level.png Frame düzeyinde TN/FP/FN/TP dağılımı
test_roc_curve_frame_level.png Frame düzeyinde eşikten bağımsız ROC performansı
test_precision_recall_curve_frame_level.png Fake sınıfı için precision-recall dengesi
test_probability_distribution_frame_level.png Real/fake olasılık dağılımlarının örtüşmesi
test_confusion_matrix_video_level.png Video düzeyindeki sınıflandırma hataları

test_roc_curve_video_level.png Video düzeyinde ROC-AUC
test_precision_recall_curve_video_level.png Video düzeyinde average precision
ablation_performance_comparison.png Birden fazla mod çalıştırılırsa accuracy/F1/AUC kıyası
save_figure fonksiyonu figürleri 150 DPI ve tight bounding box ile kaydeder, ardından figürü kapatır. Final kalite kontrolünde her PNG'nin kısa kenarının en az 600 piksel olduğu doğrulanır.

19. Kalite kapıları ve çıktı manifesti

Son blok deney tamamlanmadan önce çıktıların varlığını ve yapısal bütünlüğünü kontrol eder.

Kalite kapısı	Kontrol
required_outputs	Zorunlu checkpoint, log, metric, prediction, figure ve artifact dosyaları mevcut mu?
checkpoint_integrity	Best checkpoint model/optimizer/scheduler/scaler durumlarını içeriyor mu?
numeric_metrics	Epoch metriklerinde NaN veya Inf var mı?
prediction_uniqueness	Test sample_id değerleri benzersiz mi?
figure_resolution	Tüm figürlerin kısa kenarı en az 600 px mi?
metadata_video_leakage	Önceki video sızıntısı kontrolleri başarıyla geçti mi?

run_summary.json; model adı, özellik sayıları, frame/video metrikleri, veri muhasebesi, kalite kapıları ve süreyi tek dosyada özetler. output_manifest.csv ise run klasöründeki her dosyanın göreli yolunu, boyutunu, değiştirilme zamanını ve SHA-256 hash'ini içerir. Manifest kendi hash'ini içermez; bu döngüsel bağımlılığı önlemek için bilinçli bir tercihtir.

20. Ablation çalışma seçenekleri

run_all_ablations=False olduğunda yalnız active_mode=full çalışır. True yapılırsa altı farklı temsil kombinasyonu sırayla eğitilir ve her birinin çıktıları artifacts/ablations/<mode>/ altında tutulur. Ablation deneyinin amacı, performans artışının hangi bileşenden geldiğini ayırmaktır. Örneğin swin_only ile full karşılaştırması, 240 el yapımı özelliğin ek katkısını; texture_only ile full karşılaştırması ise Swin kolunun katkısını gösterir.

Uygulama sınırlılığı

Mevcut notebookta run_all_ablations=True olduğunda ana eğitimlerin tamamı alt klasörlere yazılır; ancak sonraki frame/video grafik ve final kalite blokları kök metrics/predictions/checkpoints dosyalarını beklemeye devam eder. Bu nedenle tüm ablationları tek geçişte çalıştırma seçeneği, sonraki raporlama hücreleri uyarlanmadan hata verebilir. Güvenli kullanım varsayılan tek-model modudur.

21. Teknik güçlü yönler

- ☐ Video bazlı split sızıntısını açık ve çok katmanlı biçimde kontrol etmesi.
- ☐ Scaler'ı yalnızca train verisinde fit ederek istatistiksel veri sızıntısını önlemesi.
- ☐ Feature çıkarımını cache'leyerek uzun işlemlerin tekrarını azaltması.
- ☐ Atomik yazma ve geri okuma kontrolleriyle checkpoint/artifact bozulmasına karşı dayanıklılık.
- ☐ Config, ortam, paket sürümleri, log, manifest ve hash üretmesi sayesinde yüksek izlenebilirlik.
- ☐ Model seçimini validation ROC-AUC, karar eşiğini validation F1 üzerinden ayırması.
- ☐ Frame ve video düzeyinde ayrı değerlendirme ve görsel raporlama üretmesi.
- ☐ Smoke test ve final quality gate yaklaşımıyla hataları erken ve geç aşamada yakalaması.
- ☐ El yapımı ve derin özellikleri aynı ortak gizli uzayda birleştiren açık, modüler mimari.

22. Sınırlılıklar ve geliştirme önerileri

Konu	Öneri
Gerçek resume mekanizması	resume_run_id yanında last.ckpt yükleme, epoch başlangıcı, optimizer/scheduler/scaler ve RNG state geri yükleme eklenmeli.
Ablation raporlama	Kök raporlama hücreleri seçilmiş ana moda yönlendirilmeli veya her mod için bağımsız tam rapor üretmeli.
Cache içerik doğrulama	Cache key içine görüntü hash'i, dosya boyutu veya mtime eklenmeli.

Video kimliği	Yalnız parent klasör adı yerine normalize tam video yolu veya yol hash'i kullanılmalı.
Padding denetimi	Siyah padding oranının sınıf/split ile korelasyonu analiz edilmeli; gerekirse reflection padding denenmeli.
Augmentation uyumu	Texture kolu için augmentation sonrası özellikleri çevrim içi hesaplama veya sadece fotometrik augmentation seçenekleri incelenmeli.
Threshold hedefi	F1 dışında balanced accuracy, Youden J veya maliyet duyarlı eşik seçimi karşılaştırılmalı.
Video agregasyonu	Mean probability; median, max/top-k, temporal attention

ve video başına kalibrasyonla kıyaslanmalı.
Kalibrasyon Probability calibration için temperature scaling veya isotonic regression değerlendirilmeli.
Tekrar deneyleri Tek seed yerine birden fazla seed ve ortalama±standart sapma raporlanmalı.
Hata analizi Manipülasyon türü, çözünürlük, blur, ROI boyutu ve padding oranına göre alt grup performansı çıkarılmalı.

23. Hücre bazlı özet

Kod bloğu	Başlık	Görev
0	Colab setup	local source bağlama ve bağımlılık kurulumu
1	local source path preflight	Gerekli klasör/dosya varlık kontrolü
2	Configuration	Tüm deney ayarları
3	Imports/repro/run	Seed, cihaz, run klasörü, log ve ortam dosyaları
4	Atomic I/O	Güvenli JSON/CSV/joblib/checkpoint yazımı
5	Metadata	Normalize, eşleştir, video ID çıkar, sızıntı kontrolü
6	Dataset	dağılımı Sınıf/split grafiği
7	Feature extraction	LBP, GLCM, Gabor, Wavelet ve 240D doğrulaması
8	Feature cache	Resume edilebilir özellik önbelleği
9	Scaler/artifacts	Train-only StandardScaler ve NPZ dosyaları
10	Dataset/loaders	Görüntü+texture örnekleri ve güvenli batch mantığı

Kod bloğu	Başlık	Görev
11	Fusion model	Swin ve texture kolları, MLP classifier
12	Metrics/helpers	Metrikler, eşik taraması, optimizer, checkpoint state
13	Train/predict loops	AMP eğitim döngüsü ve test kayıtları
14	Smoke test	Kısa uçtan uca çalışma testi
15	Train experiment	Fine-tuning, early stopping, best model ve frame test
16	Run modes	Tek model veya ablationların çalıştırılması
17	Frame figures	Loss/F1/CM/ROC/PR/olasılık grafikleri
18	Video evaluation	Mean probability agregasyonu ve video grafikleri
19	Ablation figure	Modların performans karşılaştırması
20	Final QA/manifest	Kalite kapıları, run summary ve SHA-256 manifest

24. Sonuç

Notebook, kaş ROI'lerinden deepfake tespiti yapmak için hazırlanmış, deney izlenebilirliği ve veri sızıntısı kontrolü güçlü bir hibrit makine öğrenmesi pipeline'ıdır. Kod, 240 el yapımı texture özelliğini Swin V2-Tiny derin görüntü temsilinin yanında kullanır; validation tabanlı model/eşik seçimi, frame ve video düzeyi değerlendirme, güvenli checkpointleme ve kapsamlı çıktı üretimi sağlar.

En önemli tasarım kazanımı, yalnızca model eğitmekle kalmayıp verinin doğruluğunu, deney ortamını, ara özellikleri, tahminleri, grafikleri ve dosya bütünlüğünü birlikte kaydetmesidir. Buna karşılık gerçek checkpoint resume özelliği, ablation sonrası kök raporlama uyumu ve cache içerik doğrulaması geliştirilmesi gereken başlıca alanlardır.

Bu haliyle varsayılan full modda tek bir deney çalıştırmak için kapsamlı ve güvenli bir temel sunar; çoklu ablation ve uzun süreli yeniden başlatılabilir araştırma akışları için birkaç yapısal iyileştirme önerilir.

Ek A - İnceleme yöntemi

Rapor, notebookun 1 markdown ve 21 kod hücresinin kaynak içeriği incelenerek hazırlanmıştır. Hücrelerde execution_count ve kayıtlı outputs bulunmadığından çalışma zamanı çıktıları rapora eklenmemiştir. Açıklamalar kodda fiilen bulunan fonksiyonlar, parametreler, kontrol akışları ve dosya yollarıyla sınırlandırılmıştır.